Bases de Datos NoSQL – Semana 1

Evidencia E01 · Caso integrador: Unidad de Emprendimiento del SENA

María Paula Herrera Rivas · ADSO – SENA CTMA · Instructor: Wilson Castro Gil

Medellín, octubre de 2026

Introducción

En este documento consolido la evidencia E01 de la primera semana de Bases de Datos NoSQL. Trabajé sobre el caso de la Unidad de Emprendimiento del SENA, que es una simulación: los emprendedores, iniciativas y asesorías son ficticios y las reglas son convenciones del ejercicio.

El documento incluye la comparación de modelos de datos (A2), las seis consultas y las reglas del caso (A3), el registro del entorno con la exploración del documento de asesoría (A4) y la preparación de la socialización (A5). Los scripts y documentos también están en el repositorio del trabajo (carpetas `docs/`, `scripts/` y `evidencias/`).

El diagnóstico individual DGN-01 va en su propio documento y también está en el repositorio, en la carpeta `diagnostico_nosql/`. Como trabajé sola, decidí tener todo en un mismo lugar.

A2. Comparación de necesidades y familias NoSQL

Para cada caso pensé primero en **cómo se va a consultar la información** y después en el modelo. Como dice la guía, no se trata de escoger la tecnología de moda sino la que encaja con lo que se consulta junto, lo que cambia junto y cuánto crece.

Matriz de selección

Caso	Modelo propuesto	Razón vinculada a una consulta	Límite o costo	Alternativa
Iniciativas de varios sectores	Documental (MongoDB)	Para la consulta “ver el detalle de una iniciativa con su propuesta de valor y sus atributos específicos” (Q02) puedo traer todo en un solo documento: los campos comunes (código, nombre, sector, etapa, estado) más un subdocumento con lo propio del sector, sin tener que crear columnas vacías para los demás sectores.	La flexibilidad no es gratis: si no defino un contrato para los campos comunes, cada quien puede escribir los campos distinto (`sector: "Tecnología"` vs `"tecnologia"`). Habría que agregar validación. Además, cruzar con emprendedores o asesorías ($lookup) es más costoso que un JOIN bien indexado.	SQL: una tabla `iniciativas` con los campos comunes y tablas por sector, o una columna JSON (por ejemplo JSONB en PostgreSQL) para los atributos específicos.

Caso	Modelo propuesto	Razón vinculada a una consulta	Límite o costo	Alternativa
Sesiones temporales de un portal	Clave-valor (Redis)	La consulta es siempre la misma: "dado el token de la sesión, ¿quién es el usuario y sigue vigente?". Es un acceso por clave exacta, muy rápido, y con un tiempo de expiración (TTL) la sesión se borra sola cuando vence.	Solo puedo buscar por la clave; no sirve para preguntas como "¿cuántas sesiones abrió un usuario este mes?". Los datos viven en memoria, así que si el servidor se reinicia sin persistencia se pierden (para sesiones es aceptable, para un historial no). Es un motor más que mantener.	Tabla `sesiones` en SQL con `fecha_expiracion` y una tarea que limpie las vencidas, o una colección en MongoDB con índice TTL.
Recorridos entre emprendedores, mentores y habilidades	Grafos (Neo4j)	Una consulta como "¿qué mentores tienen la habilidad *marketing digital* y ya asesoraron a emprendedores conectados con este emprendedor?" es recorrer varias relaciones seguidas. En un grafo las relaciones son parte del dato y se recorren directamente, sin encadenar muchos JOIN.	Hay que aprender otro lenguaje (Cypher) y operar otro motor. Para reportes de conteos masivos no es lo más práctico, y si las relaciones son pocas y de un solo salto, un grafo es exagerado.	SQL con tablas intermedias (`mentor_habilidad`, `asesoria`) y consultas recursivas (`WITH RECURSIVE`) cuando los saltos son pocos.
Lecturas de sensores por dispositivo y periodo (agroindustria a gran escala)	Familias de columnas (Cassandra)	La consulta prevista es "todas las lecturas del sensor S en el día D, ordenadas por hora". Puedo definir la partición por (dispositivo, día) y ordenar dentro de ella por fecha y hora, así cada consulta lee una sola partición aunque lleguen millones de lecturas.	Hay que diseñar una tabla por cada consulta: si después piden algo no previsto (por ejemplo "todas las lecturas mayores a 40 °C de cualquier sensor"), es muy difícil. No hay JOIN y montar un clúster es complejo para un proyecto pequeño.	Una base de series de tiempo (por ejemplo TimescaleDB) o una colección *time series* de MongoDB si el volumen no es tan grande.

Una situación en la que conservaría SQL

El registro de **asistencia de aprendices a talleres de la Unidad**, donde cada registro tiene siempre los mismos campos (aprendiz, taller, fecha, asistió), hay muchas relaciones fijas (aprendiz–ficha–taller) y se necesitan reportes con JOIN y totales exactos. Ahí la estructura no varía, la integridad referencial es importante y SQL resuelve bien todo eso. Es parecido a lo que hice en el sistema de Gestión de Aprendices con PHP y MySQL: los datos eran muy regulares y las tablas relacionadas funcionaban perfecto.

Práctica de la sección 5.1

Dos razones para evaluar documentos en el caso de la Unidad de Emprendimiento:

1. Las iniciativas tienen atributos distintos según el sector (una de alimentos tiene producto y canal de venta; una de software, servicio digital y forma de entrega). En documentos cada iniciativa guarda solo lo que le aplica, sin columnas vacías.
2. Varias consultas piden ver la información junta (el detalle de una iniciativa, o una asesoría con los datos básicos de la iniciativa atendida). Un documento puede traer esa información en una sola lectura.

Una razón para preferir SQL bajo otros requisitos: si la Unidad necesitara operaciones que deben cumplirse todas o ninguna sobre varias tablas a la vez (por ejemplo, manejo de recursos o convocatorias con montos) y reportes muy cruzados entre muchas entidades, la integridad referencial y las transacciones de SQL serían más seguras y sencillas de mantener.

> La flexibilidad no elimina la necesidad de modelar: aunque MongoDB no me obligue a tener un esquema, yo sí tengo que decidir qué campos son comunes, cuáles son obligatorios y qué valores son válidos.

A3. Reglas y consultas de la Unidad de Emprendimiento

Antes de pensar en colecciones o comandos, escribí las seis consultas en lenguaje natural y definí qué entra y qué sale de cada una. Las frecuencias son **hipótesis** mías sobre cómo se usaría el sistema, no estadísticas medidas.

Consultas Q01–Q06

ID	Consulta en lenguaje natural	Actor	Filtros (entrada)	Campos de salida	Orden	Frecuencia estimada (hipótesis)
Q01	Mostrar las iniciativas activas de un sector y una etapa didáctica.	Responsable de la unidad / asesor	`estado = "activa"`, `sector` (ej. `tecnologia`), `etapa` (ej. `validacion`). Sector y etapa son opcionales: si no se envían, se muestran todas las activas.	código, nombre, sector, etapa, código y alias del emprendedor responsable	sector, luego etapa (idea → validacion → puesta_en_marcha), luego nombre A-Z	Varias veces al día
Q02	Ver el detalle completo de una iniciativa, con su propuesta de valor y sus atributos específicos del sector.	Asesor (antes de una asesoría)	`codigo` de la iniciativa (ej. `INI-001`)	todos los campos comunes, `propuesta_valor`, atributos específicos del sector, código y alias del responsable, estado	No aplica (devuelve una sola iniciativa)	Alta: cada vez que se prepara una asesoría
Q03	Listar las iniciativas de un emprendedor responsable, filtrando por estado.	Responsable de la unidad	`codigo` del emprendedor (ej. `EMP-003`), `estado` opcional (`activa` / `archivada`)	código, nombre, sector, etapa, estado, fecha de registro	estado (activas primero), luego fecha de registro más reciente	Media: algunas veces por semana

ID	Consulta en lenguaje natural	Actor	Filtros (entrada)	Campos de salida	Orden	Frecuencia estimada (hipótesis)
Q04	Consultar las asesorías programadas pendientes en un rango de fechas y una modalidad.	Asesor / responsable de agenda	`estado = "programada"`, `fecha_programada` desde (incluida) hasta (excluida), `modalidad` (`virtual` / `presencial`). Las fechas del rango se interpretan en hora de Colombia y se guardan en UTC.	código de la asesoría, fecha y hora programada, modalidad, código del asesor, código y nombre de la iniciativa	`fecha_programada` ascendente (la más próxima primero)	Diaria (al iniciar la jornada)
Q05	Ver el historial de asesorías de una iniciativa.	Asesor / responsable de la unidad	`codigo` de la iniciativa. Incluye todas las asesorías (programadas, realizadas y canceladas) aunque la iniciativa esté archivada.	código de la asesoría, estado, fecha programada, fecha de realización, temas, modalidad, código del asesor	`fecha_programada` ascendente (orden cronológico)	Media: antes de cada asesoría de seguimiento
Q06	Contar cuántas asesorías se realizaron por sector de la iniciativa y por mes.	Responsable de la unidad (reporte)	`estado = "realizada"`, año (ej. 2026). El mes se calcula con `fecha_realizacion` en hora de Colombia.	sector, mes (AAAA-MM), cantidad de asesorías	mes ascendente, luego sector	Mensual (informe de gestión)

Decisión sobre el sector en Q06

Decidí que en Q06 el sector debe ser **el que tenía la iniciativa al momento de la asesoría**, no el valor actual. Si una iniciativa empezó en `alimentos` y después se reclasifica como `economia_circular`, las asesorías que ya se hicieron deben seguir contando para `alimentos`; si no, el informe de un mes que ya pasó cambiaría cada vez que alguien edita una iniciativa. Esto implica que, al modelar en S02, la asesoría probablemente debe guardar una copia (snapshot) del sector. Lo dejo planteado para retomarlo en la semana 2.

Reglas del caso

#	Regla	¿La ajusto? Justificación
R1	Los códigos de emprendedores, iniciativas y asesorías son únicos. Formato del ejercicio: `EMP-001`, `INI-001`, `ASE-001`.	Sin ajuste. Agrego el formato para que sea fácil reconocer a qué entidad pertenece un código.
R2	Cada iniciativa tiene **un** emprendedor responsable que debe existir. Una persona puede ser responsable de varias iniciativas.	Sin ajuste. Los equipos de varios socios quedan para una versión posterior, como dice la guía.
R3	Cada asesoría se vincula con una iniciativa existente y registra el código ficticio del asesor (ej. `ASR-01`).	Sin ajuste. El asesor se identifica solo por código, sin datos de contacto.
R4	Sectores válidos: `tecnologia`, `alimentos`, `economia_circular`. Etapas didácticas: `idea`, `validacion`, `puesta_en_marcha`.	Ajuste: se escriben siempre en minúscula, sin tildes y con guion bajo, para que `"Tecnología"` y `"tecnologia"` no cuenten como sectores distintos. No son categorías oficiales del SENA.

#	Regla	¿La ajusto? Justificación
R5	Estado de iniciativa: `activa` o `archivada`. Estado de asesoría: `programada`, `realizada` o `cancelada`.	Sin ajuste.
R6	Toda asesoría tiene `fecha_programada`. Cuando se realiza, se registra `fecha_realizacion`, que puede ser distinta de la programada. La fecha programada no se modifica para ocultar una reprogramación.	Ajuste: mientras la asesoría está `programada` o `cancelada`, `fecha_realizacion` es `null` (no se deja ausente ni como cadena vacía); si está `realizada`, `fecha_realizacion` es obligatoria y de tipo fecha.
R7	No se programan dos asesorías del mismo asesor en la misma franja.	Ajuste: defino la **franja del laboratorio** como bloques de 1 hora que empiezan en punto (08:00–09:00, 09:00–10:00, … en hora de Colombia). Una asesoría ocupa la franja en la que empieza su `fecha_programada`. Las asesorías `canceladas` no ocupan franja, para que se pueda volver a usar ese espacio.
R8	Archivar una iniciativa no elimina su historial de asesorías.	Sin ajuste. Archivar es un cambio de estado, no un borrado; el historial (Q05) se sigue pudiendo consultar.

Reglas funcionales vs. mecanismos

Las reglas de arriba dicen **qué** debe cumplirse y se pueden explicar sin una pantalla ni un motor concreto. El **cómo** garantizarlas es lo que vamos a aprender más adelante:

Regla funcional	Posible mecanismo (a estudiar después)
Códigos únicos (R1)	Usar el código como `_id` o un índice único
Responsable / iniciativa existentes (R2, R3)	Validación en la aplicación antes de insertar; MongoDB no tiene claves foráneas como SQL
Valores permitidos de sector, etapa, estado, modalidad (R4, R5)	Validación de esquema (`$jsonSchema`) con listas de valores (`enum`)
Coherencia de fechas según estado (R6)	Validación de esquema y reglas en la aplicación
Un asesor no puede tener dos asesorías en la misma franja (R7)	Índice único compuesto (asesor + franja) para asesorías no canceladas. Comprobar antes de insertar **no basta** si dos personas programan al mismo tiempo; esa garantía concurrente se verá más adelante.
Archivar no borra el historial (R8)	Cambiar el campo `estado` en lugar de usar `deleteOne` / `deleteMany`

Dos preguntas para la persona responsable de la unidad

1. ¿Una asesoría puede atender **a varias iniciativas a la vez** (por ejemplo, un taller grupal), o siempre es una iniciativa por asesoría? Esto cambia si la asesoría guarda una iniciativa o una lista.
2. Cuando una asesoría se **reprograma**, ¿prefieren cancelarla y crear una nueva, o cambiar la fecha guardando un historial de cambios? Y ¿cuánto dura normalmente una asesoría? Necesito saberlo para confirmar si la franja de 1 hora es correcta.

Riesgo identificado

Doble reserva de un asesor: si dos personas programan al mismo asesor en la misma franja casi al mismo tiempo, las dos pueden ver la franja libre y guardar la asesoría, quedando el asesor con dos citas cruzadas. Revisar antes de insertar no evita este caso. Mientras se aprende el mecanismo adecuado, la mitigación es que una sola persona maneje la agenda de cada asesor y que exista un reporte que detecte franjas duplicadas.

También veo un riesgo de **calidad de datos**: si los sectores, etapas o modalidades se escriben a mano sin validación, los reportes como Q06 pueden salir mal (sectores repetidos con distinta escritura).

A4. Registro del entorno y exploración de un documento

> Este registro no contiene contraseñas. La contraseña de la cuenta de práctica solo se escribe en el prompt de mongosh o en el campo protegido de Compass.

1. Sistema y versiones

Elemento	Versión / dato	Cómo lo verifiqué
Sistema operativo	Windows	Equipo personal
Python	3.14.6	`python --version`
Git	2.51.0.windows.1	`git --version`
mongosh	2.13.0	`mongosh --version`
Servidor MongoDB Community	9.0.2 (local, instalado como servicio de Windows)	Dentro de mongosh: `db.version()`
MongoDB Compass	No instalado	La verificación la hice en mongosh, como permite la guía
Base de datos	`emprendimiento_sena_lab`	`db.getName()`
Usuario de práctica	`aprendiz_nosql` (sin contraseña en este registro)	Lo creé yo con rol `readWrite` solo sobre `emprendimiento_sena_lab` (ver sección 1.1)
Base de autenticación	`emprendimiento_sena_lab`	Datos de la cuenta

Si el instructor entrega otro nombre de base o de usuario, lo cambio aquí y en todos los comandos.

1.1 Instalación y cuenta de práctica

Como no tenía una instancia preparada, instalé MongoDB en mi computador. Desde PowerShell **como administrador**:

```
winget install MongoDB.Server
winget install MongoDB.Shell
```

Después creé la cuenta de práctica, con permisos solo de lectura y escritura sobre la base del laboratorio (no es una cuenta administrativa). La contraseña se pide con `passwordPrompt()`, así que no queda escrita en el comando ni en el historial:

```
use emprendimiento_sena_lab
db.createUser({ user: "aprendiz_nosql", pwd: passwordPrompt(), roles: [ {
role: "readWrite", db: "emprendimiento_sena_lab" } ] })
```

Resultado: `{ ok: 1 }`.

2. Conexión autenticada

En PowerShell el comando va en una sola línea (en Windows no se usa \ para partir líneas):

```
mongosh "mongodb://127.0.0.1:27017/emprendimiento_sena_lab" --username
aprendiz_nosql --authenticationDatabase emprendimiento_sena_lab
```

La contraseña la escribo cuando mongosh la pide (no se ve mientras se escribe). Dentro de mongosh:

```
db = db.getSiblingDB("emprendimiento_sena_lab");
db.runCommand({ping: 1});
db.getName();
```

Resultado esperado: `{ ok: 1 }` y `emprendimiento_sena_lab`. **Resultado obtenido:**

```
emprendimiento_sena_lab> db.runCommand({ping: 1});
{ ok: 1 }
emprendimiento_sena_lab> db.getName();
emprendimiento_sena_lab
```

Al conectar, el servidor mostró esta advertencia: *"Access control is not enabled for the database"*. Significa que en mi instalación local el control de acceso no está activado: me conecté autenticada con `aprendiz_nosql`, pero el servidor también aceptaría conexiones sin usuario. Para esta práctica local no afecta los resultados, pero en un entorno real (como el que prepara el instructor) la autenticación debe estar obligatoria. Activarla implica crear primero un usuario administrador y habilitar `security.authorization` en el archivo `mongod.cfg`; lo dejo como mejora pendiente y no desactivé ni cambié nada de seguridad para "arreglar" la advertencia.

El `ping` solo demuestra que hay conexión; el permiso de lectura y escritura lo compruebo con el script de exploración.

3. Exploración del documento (`scripts/01_explorar.js`)

```
mongosh "mongodb://127.0.0.1:27017/emprendimiento_sena_lab" --username
aprendiz_nosql --authenticationDatabase emprendimiento_sena_lab --file
scripts/01_explorar.js
```

Resultado esperado: se muestra el documento `ASE-DEMO-001`, el conteo da `1` y la comprobación de fecha da `true`. Si lo ejecuto otra vez, el conteo sigue en `1` porque el script revisa antes de insertar. **Resultado obtenido** (también guardado en `evidencias/02_explorar_resultado.txt`):

```
{
  _id: 'ASE-DEMO-001',
  emprendedor_alias: 'Emprendedor ficticio 01',
  iniciativa: { codigo: 'INI-DEMO-001', nombre: 'EcoEmpaque', sector:
'economia_circular' },
  temas: [ 'propuesta de valor', 'validacion de clientes' ],
  modalidad: 'virtual',
  estado: 'programada',
  requiere_seguimiento: true,
  fecha_programada: ISODate('2026-10-08T13:00:00.000Z'),
  fecha_realizacion: null
}
```

```
1
true
```

Coincide con lo esperado. Un detalle que noté: mongosh muestra la fecha como `ISODate(...)`, que es la forma en que presenta un valor de tipo Date, y la comprobación `instanceof Date` dio `true`.

Tipos del documento

Campo	Valor de ejemplo	Tipo	Explicación
`_id`	`"ASE-DEMO-001"`	Cadena (string)	Identificador único dentro de la colección. Es una cadena estable, no un ObjectId; no es contraseña ni da permisos.
`emprendedor_alias`	`"Emprendedor ficticio 01"`	Cadena	Alias ficticio, sin datos personales reales.
`iniciativa`	`{ codigo, nombre, sector }`	**Subdocumento**	Objeto dentro del documento con los datos básicos de la iniciativa atendida.
`iniciativa.sector`	`"economia_circular"`	Cadena	Uno de los sectores del ejercicio.
`temas`	`["propuesta de valor", "validacion de clientes"]`	**Arreglo**	Lista corta y acotada de temas tratados.
`modalidad`	`"virtual"`	Cadena	Nada en el documento obliga a que sea `virtual` o `presencial`; eso es una regla que hay que validar aparte.
`estado`	`"programada"`	Cadena	Estado de la asesoría.
`requiere_seguimiento`	`true`	**Booleano**	Es `true` sin comillas. No es lo mismo que la cadena `"true"`.
`fecha_programada`	`new Date("2026-10-08T13:00:00Z")`	**Date**	Instante en UTC; en Colombia corresponde a las 08:00 del 8 de octubre. Se comprueba con `instanceof Date` → `true`.
`fecha_realizacion`	`null`	Nulo	Nulo explícito: según nuestra regla R6, significa que la asesoría todavía no se ha realizado.

4. Compass

Configuré la conexión con host `127.0.0.1`, puerto `27017`, usuario `aprendiz_nosql`, base de autenticación `emprendimiento_sena_lab` y la contraseña en su campo protegido. Luego ubiqué `emprendimiento_sena_lab` → `asesorias_demo` → documento `ASE-DEMO-001`. **Resultado:** no instalé Compass en mi equipo, así que hice la verificación en mongosh (sección 3): localicé la base `emprendimiento_sena_lab`, la colección `asesorias_demo` y el documento `ASE-DEMO-001`. Si lo instalo después (`winget install MongoDB.Compass.Community`), los datos de conexión son los de arriba.

No comparto la URI con la contraseña incluida.

5. ¿Por qué esta demostración todavía no es el modelo definitivo?

- La colección `asesorias_demo` es solo para experimentar; en S02 se van a decidir las colecciones reales `emprendedores`, `iniciativas` y `asesorias`.

- El documento **copia** el alias del emprendedor y los datos de la iniciativa dentro de la asesoría. Si la iniciativa cambia de nombre, habría que actualizar todas sus asesorías. Todavía no se ha decidido qué se referencia, qué se embebe y qué se guarda como copia histórica (por ejemplo, el sector para Q06).
- Le falta el **código del asesor**, que es necesario para la regla de franjas (R7).
- No hay **validación**: nada impide escribir una modalidad o un estado que no exista, o dejar la fecha como texto.

6. Ejercicio de transferencia (`scripts/02_transferencia.js`)

En una copia del ejemplo cambié `_id` a `ASE-DEMO-002`, la iniciativa a `INI-DEMO-002`, el nombre a `Sabores Locales` y el sector a `alimentos`.

Predicción antes de ejecutar: se inserta un documento nuevo; el conteo de `ASE-DEMO-002` da `1` y el de `ASE-DEMO-001` sigue en `1`; la fecha sigue siendo `Date` (`true`). Si lo ejecuto dos veces, no se duplica.

Qué cambia y qué se conserva: cambian los **valores** de `_id`, `iniciativa.codigo`, `iniciativa.nombre` e `iniciativa.sector`, pero todos siguen siendo cadenas. Los demás campos se conservan igual y con el mismo tipo: `temas` sigue siendo arreglo, `requiere_seguimiento` booleano, `fecha_programada` Date y `fecha_realizacion` null.

Resultado obtenido (también en `evidencias/03_transferencia_resultado.txt`):

```
{
  _id: 'ASE-DEMO-002',
  emprendedor_alias: 'Emprendedor ficticio 01',
  iniciativa: { codigo: 'INI-DEMO-002', nombre: 'Sabores Locales', sector:
'alimentos' },
  temas: [ 'propuesta de valor', 'validacion de clientes' ],
  modalidad: 'virtual',
  estado: 'programada',
  requiere_seguimiento: true,
  fecha_programada: ISODate('2026-10-08T13:00:00.000Z'),
  fecha_realizacion: null
}
Conteo ASE-DEMO-001: 1
Conteo ASE-DEMO-002: 1
fecha_programada es Date: true
tipo de iniciativa.sector: string
temas es arreglo: true
requiere_seguimiento es booleano: true
```

La predicción se cumplió: cada ID tiene un solo documento y todos los campos conservaron su tipo.

7. Reto de la sección 5.5

- **Atributo específico para una iniciativa de alimentos:** `canal_venta: ["tienda de barrio", "domicilios"]` (arreglo de cadenas), porque una misma iniciativa puede vender por varios canales.

- **Atributo específico para una iniciativa de software:** `modalidad_entrega: "aplicacion_web"` (cadena), para indicar cómo se entrega el servicio digital.
- **Regla que el objeto por sí solo no impone:** que `modalidad` pertenezca al conjunto acordado (`virtual`, `presencial`). Otra: que si `estado` es `realizada`, `fecha_realizacion` no puede ser `null`.

8. Dificultad encontrada y cómo la resolví

1. Error 1603 al instalar el servidor. La primera vez corrí `winget install MongoDB.Server` desde la terminal de VS Code; descargó todo pero terminó con *"Error del instalador con el código de salida: 1603"*. Ese código indica que el instalador no tenía permisos suficientes, porque MongoDB se instala como servicio de Windows. Lo solucioné abriendo PowerShell con **"Ejecutar como administrador"** y repitiendo el comando: *"Instalado correctamente"*.

2. `mongosh` no se reconocía. Después de instalarlo, la terminal decía *"El término 'mongosh' no se reconoce como nombre de un cmdlet…"*. El programa sí estaba instalado, pero la terminal todavía tenía la lista de rutas (PATH) anterior. Lo resolví recargando el PATH en la sesión:

```
$env:Path = [System.Environment]::GetEnvironmentVariable("Path","Machine") +
";" + [System.Environment]::GetEnvironmentVariable("Path","User")
```

Después, `mongosh --version` mostró `2.13.0`.

3. Comandos en varias líneas. En la guía la conexión aparece partida en varias líneas con `\`, pero eso es sintaxis de Linux/macOS. En PowerShell lo escribí en una sola línea.

A5. Socialización de E01 y nivelación

Lo que presento

Elemento	Mi aporte
Una elección de modelo	Para las iniciativas de varios sectores elijo el modelo **documental** (MongoDB), porque la consulta Q02 trae el detalle de una iniciativa con sus atributos específicos en una sola lectura. Su límite es que la flexibilidad exige definir un contrato y validar los campos comunes.
Una consulta	**Q04:** asesorías `programadas` en un rango de fechas (desde incluido, hasta excluido) y una modalidad, ordenadas por `fecha_programada` de la más próxima a la más lejana. Entrada: rango y modalidad. Salida: código, fecha, modalidad, asesor e iniciativa.
Un tipo	`fecha_programada` es de tipo **Date**, no texto. `"2026-10-08T13:00:00Z"` entre comillas es una cadena aunque parezca fecha; con `new Date(...)` se vuelve un Date y se comprueba con `instanceof Date`. En Colombia ese instante es a las 08:00.
Una dificultad	Diferenciar **campo ausente, `null` y cadena vacía**. Entendí que `fecha_realizacion: null` puede significar "todavía no se ha realizado" solo si así lo definimos en las reglas (R6), y que no se debe usar `""` para una fecha desconocida.

Retroalimentación del compañero (lista E01)

Compañero(a)	Fortaleza que señaló	Sugerencia de mejora
Por diligenciar en la socialización		

Acción de mejora acordada

Acción concreta	Fecha acordada
Repasar consultas SQL con JOIN y practicar la diferencia entre `null`,	Antes de la sesión de la semana 2

Acción concreta	Fecha acordada
campo ausente y cadena vacía con ejemplos en Python y en mongosh.	(por confirmar con el instructor)

Uso de IA

Usé IA como apoyo en el diagnóstico DGN-01 y en esta evidencia, para organizar las matrices y revisar la redacción de las reglas y consultas. Verifiqué que cada consulta tuviera entrada, salida y orden; comprobé la sintaxis de los scripts y que al ejecutarlos dos veces no se duplicaran los documentos.

Lista de chequeo E01

Ítem observable	Estado	Evidencia / observación
Entrega diagnóstico teórico y práctico con autoevaluación	Cumple	Documento DGN-01 y carpeta diagnostico_nosql/ del repositorio
Justifica cuatro elecciones de modelo y sus límites	Cumple	Sección A2: matriz de selección y caso SQL
Formula Q01–Q06 con filtros, salidas y orden	Cumple	Sección A3: tabla de consultas y decisión de Q06
Documenta reglas, dos preguntas a la unidad y un riesgo	Cumple	Sección A3: R1–R8, reglas vs. mecanismos, preguntas y riesgo
Conecta utilizando cuenta de práctica	Cumple	entorno.md secciones 1.1 y 2: ping `{ ok: 1 }` con `aprendiz_nosql`
Explica cadena, booleano, arreglo, subdocumento y Date	Cumple	Sección A4: tabla de tipos del documento
Reproduce la exploración y registra resultados	Cumple	entorno.md secciones 3 y 6, carpeta evidencias/
Conserva versiones y pasos de ejecución	Cumple	Repositorio Git y README con pasos
Mantiene evidencias sin secretos ni datos reales	Cumple	.gitignore, sin contraseñas, solo datos ficticios